AML S26 Final Project

Jordan Miller and Sydney Williams

May 2026

Contents

0 Introduction and Acknowledgments **3**

1 Part I **3**
1.1 Data Overview . . . 3
1.1.1 File size, class balance, scale of features . . . 3
1.2 Design Decisions . . . 3
1.2.1 Pipeline elements (Traditional Method) . . . 3
1.2.2 Pipeline elements (Deep Learning Method) . . . 4
1.2.3 Model selection (Traditional) . . . 4
1.2.4 Model selection (Deep Learning) . . . 4
1.3 Traditional Results . . . 4
1.3.1 Numerical metrics . . . 5
1.3.2 Confusion matrix . . . 6
1.3.3 Predicted results on test set . . . 6
1.4 Deep Results . . . 6
1.4.1 Development and training time . . . 6
1.4.2 Numerical metrics . . . 6
1.4.3 Confusion matrix . . . 6
1.4.4 Analysis . . . 6
1.4.5 Predicted results on test set . . . 7
1.5 Conclusion . . . 7
1.5.1 Model comparison . . . 7
1.5.2 Lessons learned . . . 7

2 Part II **7**
2.1 Data Overview . . . 7
2.1.1 File Size, Class Balance, Scale of Features . . . 7
2.2 Design Decisions . . . 7
2.2.1 Pipeline Elements (Traditional Method) . . . 7
2.2.2 Pipeline Elements (Deep Learning Method) . . . 8
2.2.3 Model Selection (Traditional) . . . 8
2.2.4 Model Selection (Deep Learning) . . . 9
2.3 Results . . . 9
2.3.1 Development and Training Time . . . 9
2.3.2 Numerical Metrics . . . 10
2.3.3 Confusion Matrix . . . 10
2.3.4 Analysis . . . 10
2.3.5 Predicted Results on Test Set . . . 10
2.4 Conclusion . . . 11
2.4.1 Model Comparison . . . 11
2.4.2 Lessons Learned . . . 11

0 Introduction and Acknowledgments

The goal of this project for ECE:5995 Applied Machine Learning is to combine all knowledge acquired over the course of the semester into a comprehensive machine learning solution to two problems. The steps followed were:

- Develop a suitable classification approach for each problem utilizing a a) traditional and b) deep learning method. Use appropriate preprocessing steps as needed.
- Optimize and compare the performance of both methods based on suitable performance metrics.
- Analyze and discuss your results as well as the time required for development and training of the classifiers.

LLM AI models were used for some portions of code development, specifically formatting, error checking, and creating result summary/plotting scripts. All final design decisions, including models and preprocessing used, as well as the report were made by Jordan and Sydney.

1 Part I

1.1 Data Overview

1.1.1 File size, class balance, scale of features

The PartI_dev.csv file contains 48 features. The dataset comprises 46,805 numerical samples evenly distributed across 11 distinct classes, with exactly 4,255 samples per class. This represents a perfectly balanced dataset, eliminating the need for class weighting or resampling methods. There are no missing values.

1.2 Design Decisions

The large number of samples and perfectly balanced classes allowed for a straightforward evaluation of various models without the need for complex resampling techniques.

1.2.1 Pipeline elements (Traditional Method)

StandardScaler was utilized to normalize the features to zero-mean and unit-variance. Normalization ensures that models sensitive to feature scale, such as K-Nearest Neighbors and Logistic Regression, perform optimally.

For dimensionality reduction, Principal Component Analysis (PCA) was evaluated. A variance threshold of 95% was applied to reduce the dimensionality of the feature space. With PCA, the dimensionality was reduced from 48 predictors to 31 predictors. Both the original raw features and the PCA-transformed features were tested to determine the most predictive input space.

1.2.2 Pipeline elements (Deep Learning Method)

Similar to the traditional approach, StandardScaler was used to normalize the raw features. Since the dataset is perfectly balanced, techniques like SMOTE or class-weighted loss functions were unnecessary. The deep learning model was trained directly on the normalized raw features.

1.2.3 Model selection (Traditional)

A variety of traditional machine learning models were benchmarked, including Logistic Regression, Random Forest, Gradient Boosting, K-Nearest Neighbors (KNN), Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), and Support Vector Machine (SVM).

In order to tune the hyper parameters for some models, GridSearchCV was used for KNN and Logistic Regression. K values from 1 to 50 were tested for KNN, and 0.01, 0.1, 1, and 10 were tested as values for Regularization Strength in Logistic Regression.

While GridSearchCV could have been used to tune hyperparameters in other tested models, the time it takes to train multiple models is not worth the small improvement. Many of the chosen hyperparameters are standard for balanced classes.

To ensure robust performance estimates, StratifiedKFold cross-validation with 5 splits was used to compare the models on the raw features. The models were evaluated on the mean accuray score after 5 fold CV. Macro F1 score was also reported. Accuracy was chosen due to the perfectly balanced classes.

1.2.4 Model selection (Deep Learning)

A Multi-Layer Perceptron was selected for the deep learning approach. The network architecture consists of two hidden layers with 128 and 64 neurons, respectively. BatchNorm was included after each linear layer to normalize activations and stabilize training. ReLU was utilized as the activation function, along with Dropout(0.3) to prevent overfitting.

The model was compiled using CrossEntropyLoss, and optimized using Adam with a weight decay of 1e-4. Model performance was validated using 5-fold StratifiedKFold cross-validation.

1.3 Traditional Results

Based on the CV results (Figure 1), Gradient Boosting + PCA was chosen as the best model for traditional machine learning. Gradient Boosting had the best mean accuracy, as well as the top F1 macro score.

After selecting Gradient Boosting + PCA as the best model for the given data set, it was retrained on the whole dataset. The machine used to train it was my laptop. With an AMD Ryzen 7 running at 3.30 GHz, re-training the model took 15.36 seconds.

While Gradient Boosting was chosen as the best model, SVM + PCA and LightGBM + PCA performed very similary. LightGBM was about three times quicker to run 5-fold CV

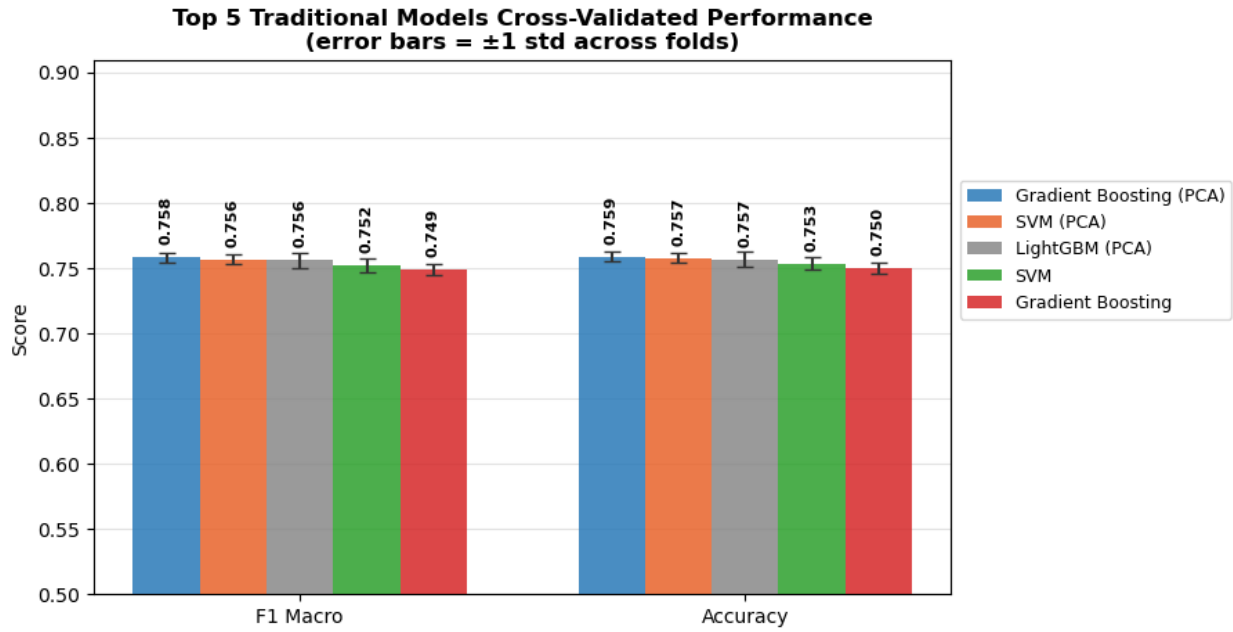

Figure 1: F1 Macro and Accuracy of Top 5 Traditional Models

on than Gradient Boosting, so if we had a ginormous data set, the time save may be worth it. But in this case, 13 seconds versus 39 seconds is not a big difference, so the very small performance gain in Gradient Boosting is worth it.

1.3.1 Numerical metrics

Figure 1 shows a detailed report of all the models trained with 5-fold CV. Graident Boosting + PCA performed the best with a mean accuracy score of 0.759, it had a macro F1 score of 0.758. Both SVM + PCA and LightGBM + PCA had an accuracy score of 0.756 and a F1 macro score of 0.757. While it cannot be seen in Figure 1, Quadratic Discriminate Analysis performed the worst with a mean accuracy score of 0.635 and a mean F1 macro score of 0.605.

1.3.2 Confusion matrix

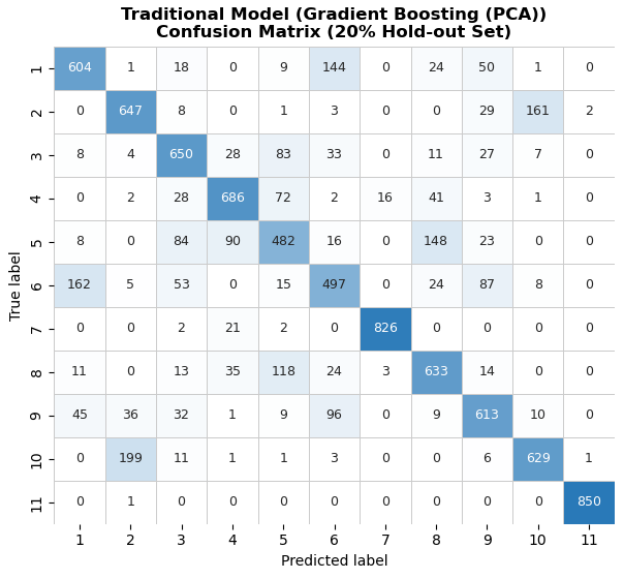

Figure 2: Confusion Matrix for GB + PCA on 20% holdout set

Figure 2 shows the confusion matrix on the 20% of data left out for testing, before it was trained on the whole dataset. There were a few common mis-classifications, with the most common being classifying class 10 as class 2. This mis-classification happened 199 times.

1.3.3 Predicted results on test set

After the best model was selected via 5-fold CV, it was tested on the 20% of data held out from the whole dataset. The model scored 0.76 for both F1 macro and accuracy.

1.4 Deep Results

1.4.1 Development and training time

[Talk about dev and training time]

1.4.2 Numerical metrics

[Talk about numerical metrics (F1, roc auc, etc)]

1.4.3 Confusion matrix

[Talk about confusion matrix]

1.4.4 Analysis

[analysis]

1.4.5 Predicted results on test set

[Talk about predicted results on test set]

1.5 Conclusion

1.5.1 Model comparison

Deep learning performed slightly better ...

1.5.2 Lessons learned

[What have you learned?]

2 Part II

2.1 Data Overview

2.1.1 File Size, Class Balance, Scale of Features

The PartII_dev.csv file contains 125 features. Each feature has 158 numerical samples, with 126 belonging to class 0 and 32 belonging to class 1. This represents a 4:1 class imbalance. The features have widely ranging scales and many feature pairs are correlated (—r—¿0.90). There are no missing values.

2.2 Design Decisions

The massive redundancy in features and very small sample size were the main constraints for this part of the project. Four iterations were tested with the final being the selected iteration of the models. Resampling and minority class synthesis techniques were experimented with, but with the dataset being so small they did not prove to be useful. The dataset being so small also caused consistent overfitting issues, meaning design decisions had to be carefully selected to minimize overfitting.

2.2.1 Pipeline Elements (Traditional Method)

StandardScalar was used in all versions of the traditional ML solution. StandardScalar normalizes the data to zero-mean and unit-variance, which was necessary due to the variety of scale in the dataset.

For dimensionality reduction, initially PCA (Principal Component Analysis) was selected. PCA discards all features except for the ones that account for 95% of the variance (in this case, all but 11 features). However, those high-variance features were not actually the most predictive features. Instead, SelectKBest was used. SelectKBest runs an ANOVA F-test on each feature independently against Y. The 10 most statistically discriminative features were kept and the rest were discarded.

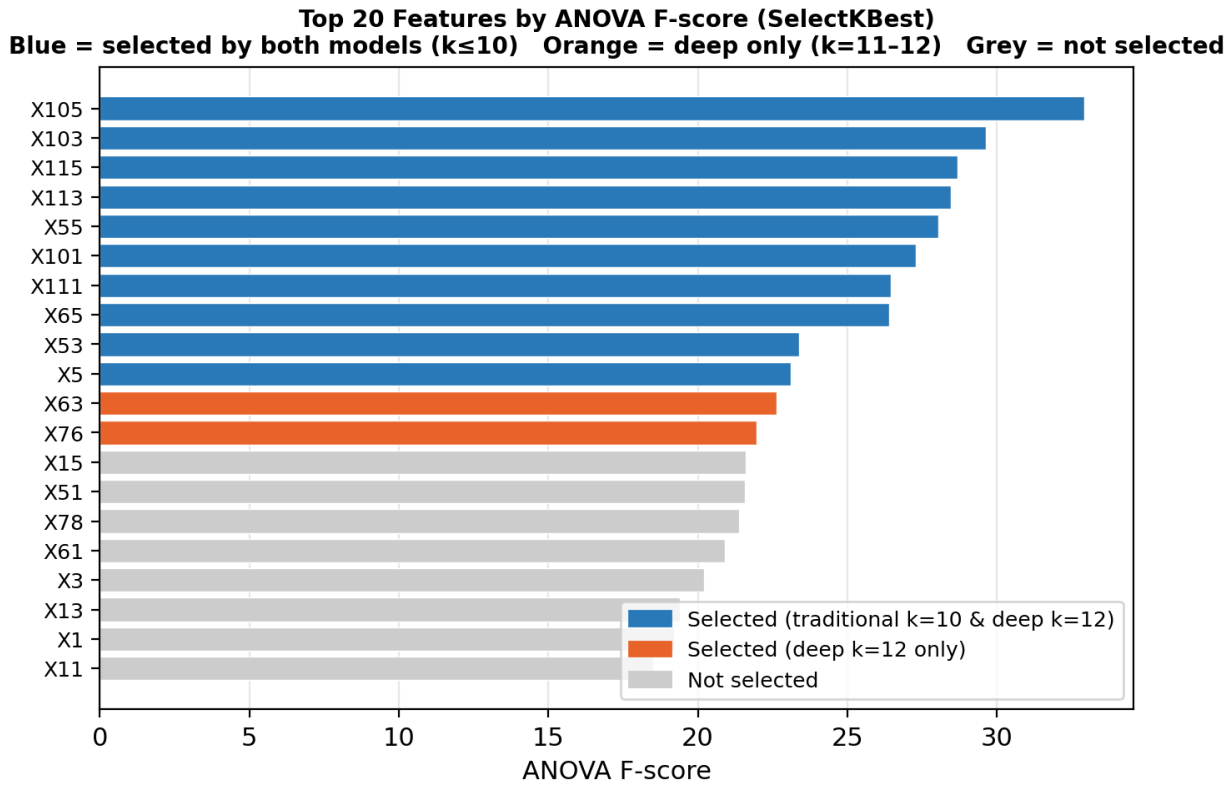

Figure 3: Comparison of features selected using SelectKBest for both Part II models

In earlier versions, SMOTE was used to generate synthetic minority-class samples. However, it did not show significant improvements in performance. LogReg can be set to class_weight(balanced) to adjust for the unbalanced classes instead.

2.2.2 Pipeline Elements (Deep Learning Method)

SelectKBest and StandardScalar are again used for the deep learning model. Like the traditional method, SMOTE was originally used. Not only did it not improve performance, it seemed to actively degrade it. In the final version there is no minority class synthesis.

2.2.3 Model Selection (Traditional)

To select a model, StratifiedKFold cross validation (specdifically RepeatedStratifiedKFold) with 5 splits repeated 5 times was used to compare models. CV F1, CV AUC-ROC, and threshold were compared between LogReg with SelectKBest(k=10), LogReg with SelectKBest(k=7), SVM using RBF kernel and SelectKBest(k=8), and LogReg with PCA.

Based on the CV results, LogReg with SelectKBest(k=10) was selected as the traditional learning method.

Model	CV F1 (macro)	CV AUC-ROC	Threshold
LogReg + SelectKBest(k=10)	**0.724 ± 0.066**	**0.738**	0.626
LogReg + SelectKBest(k=7)	0.724 ± 0.071	0.738	0.617
SVM(RBF) + SelectKBest(k=8)	0.702 ± 0.070	0.701	0.298
LogReg + PCA(95%)	0.648 ± 0.064	0.618	0.612

Figure 4: Result of model selection StratifiedKFold cross validation

2.2.4 Model Selection (Deep Learning)

MLP (Multi-Layer Perceptron) was, based on research, used for this dataset. MLP is a deep learning model well-suited for non-linear class boundaries. After experimenting with model size, it was determined to be best to use a smaller model. The original large model not only took too long to train, but was also not performant on the very small dataset.

The final model used a single hidden layer. BatchNorm was used to normalize activations to zero mean/unit variance per batch, which stabilized training and acted as a mild regularizer. BatchNorm also reduced sensitivity to weight initialization. ReLU was used as the activation function with Dropout(0.25). BCEWithLogitsLoss was used for the output layer. For the final training, the model trained for 250 epochs and used ensemble over 5 random initializations. The final model had 220 parameters.

To determine the best configurations, RepeatedStratifiedKFold CV was used for 15 folds. During these training steps, early stopping was utilized.

Config	Architecture	CV F1 (macro)	CV AUC-ROC	Avg epoch
k12_16	12→16→1	**0.758 ± 0.085**	0.706	7.9
k8_16	8→16→1	0.745 ± 0.068	0.722	6.1
k8_32_8	8→32→8→1	0.744 ± 0.069	0.705	9.0

Figure 5: Result of config selection StratifiedKFold cross validation

Based on these results, the best configuration and the one used was k12_16: 12 input features, 16 neurons in the hidden layer, 0.25 dropout (randomly zero 25% of neurons, helps prevent overfitting), weight decay of 1e-12, and learning rate of 1e-3.

2.3 Results

2.3.1 Development and Training Time

The development time for the Part II solutions were significantly longer than for the Part I solutions. This is mostly due to the large amount of experimentation that was needed to find a model that performed well with the, frankly, pretty hard to use dataset.

The machine used to train the Part II models used an NVIDIA GeForce RTX 4060 Ti and was able to make use of CUDA acceleration. The training time for the final traditional model was 20.6 s. The training time for the final, smaller neural net was 268 s (5-seed ensemble).

2.3.2 Numerical Metrics

- **Traditional:**

Metric	CV estimate	Training (full dev set)
Accuracy	–	0.7848
F1 (macro)	0.724 ± 0.066	0.6815
F1 (minority class=1)	–	0.5000
AUC-ROC	0.738	0.7562
Threshold	0.626	–
Training time	–	20.6 s

Figure 6: Numerical results of the final traditional model

- **Deep Learning:**

Metric	CV estimate	Training (full dev set)
Accuracy	–	0.7658
F1 (macro)	0.758 ± 0.085	0.6823
F1 (minority class=1)	–	0.5195
AUC-ROC	0.706	0.7904
Threshold	0.496	–
Training time	–	268 s (5-seed ensemble)

Figure 7: Numerical results of the final deep learning model

2.3.3 Confusion Matrix

2.3.4 Analysis

2.3.5 Predicted Results on Test Set

[Talk about predicted results on test set]

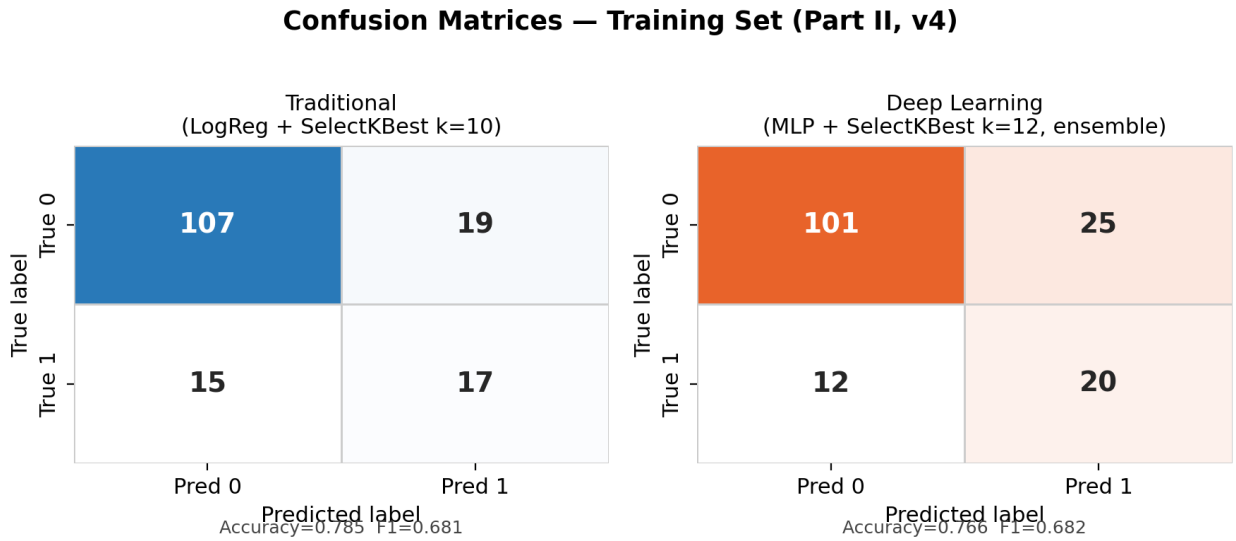

Figure 8: Confusion matrices for Part II

2.4 Conclusion

2.4.1 Model Comparison

The following figures were generated based on the results testing ona reserved development portion of the dataset of both Part II models:

2.4.2 Lessons Learned

This part of the project was an exercise in dealing with sub-optimal data. The low amount of samples made it very difficult to get performance out of the models. I also thought, based on what we learned in class, that synthetic minority class generation may help, but that's not a universal solution!

I chose to develop both models even though only one was required because I wanted more practice developing deep learning models.

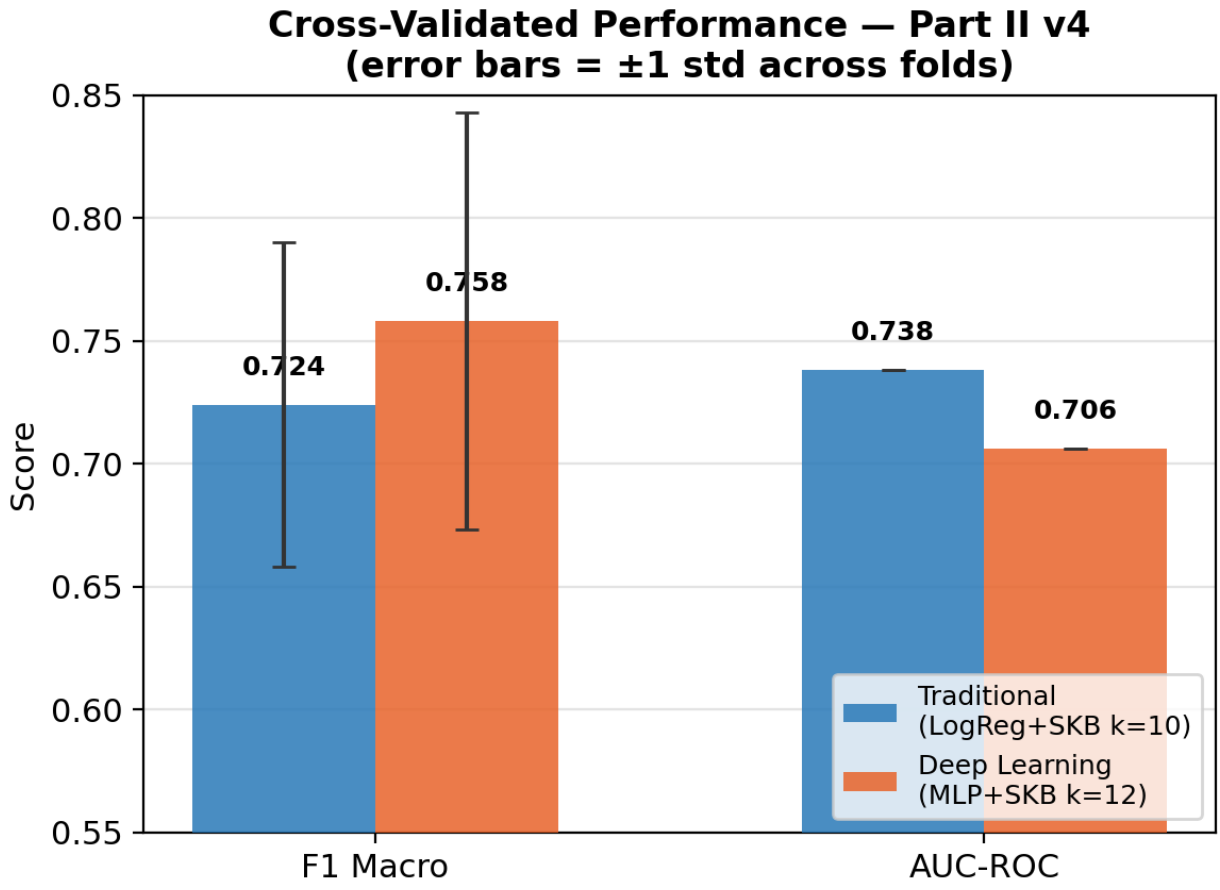

Figure 9: Comparison of CV results for macro F1 and AUC-ROC for Part II models

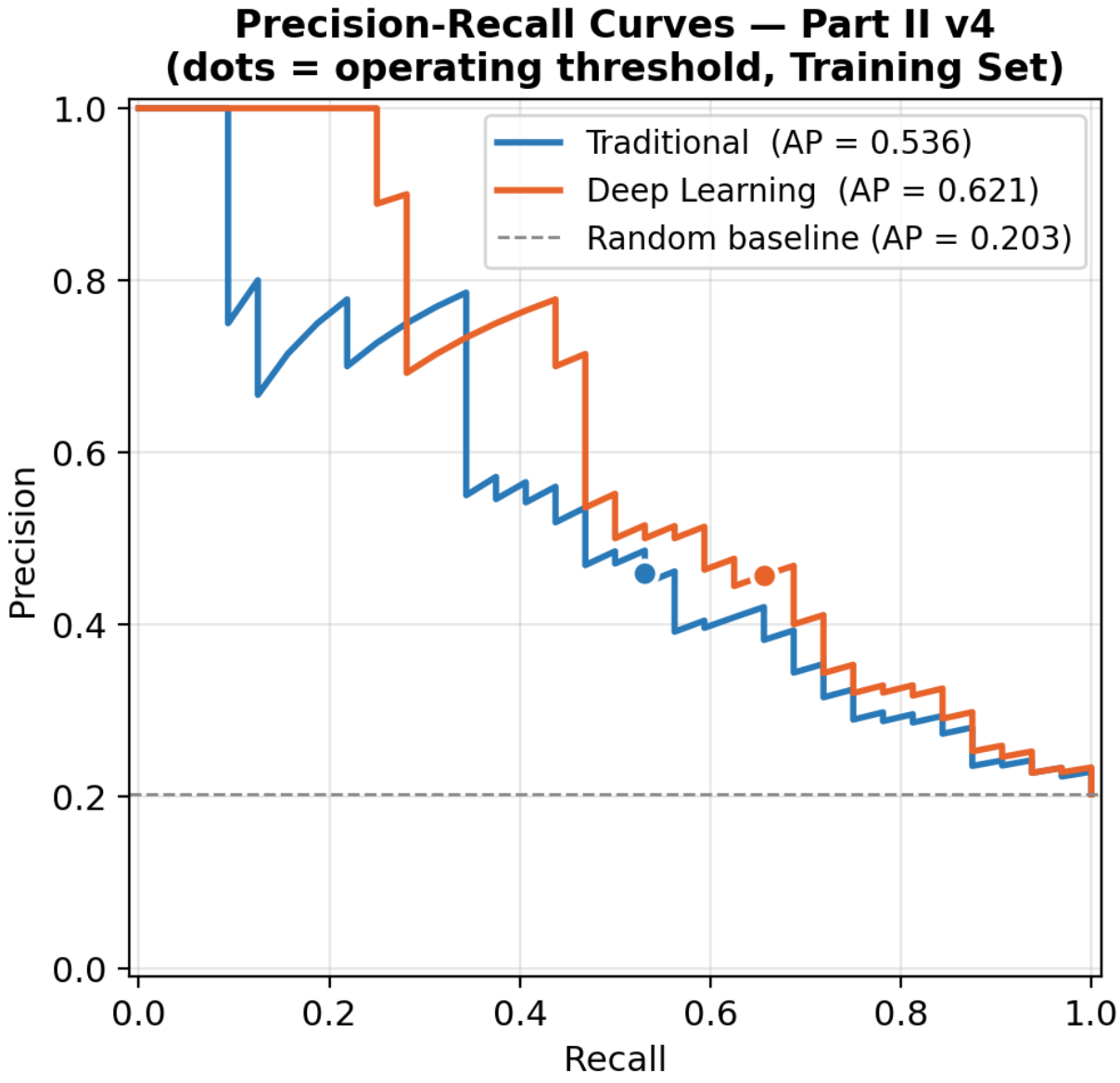

Figure 10: Comparison of precision-recall results for both Part II models

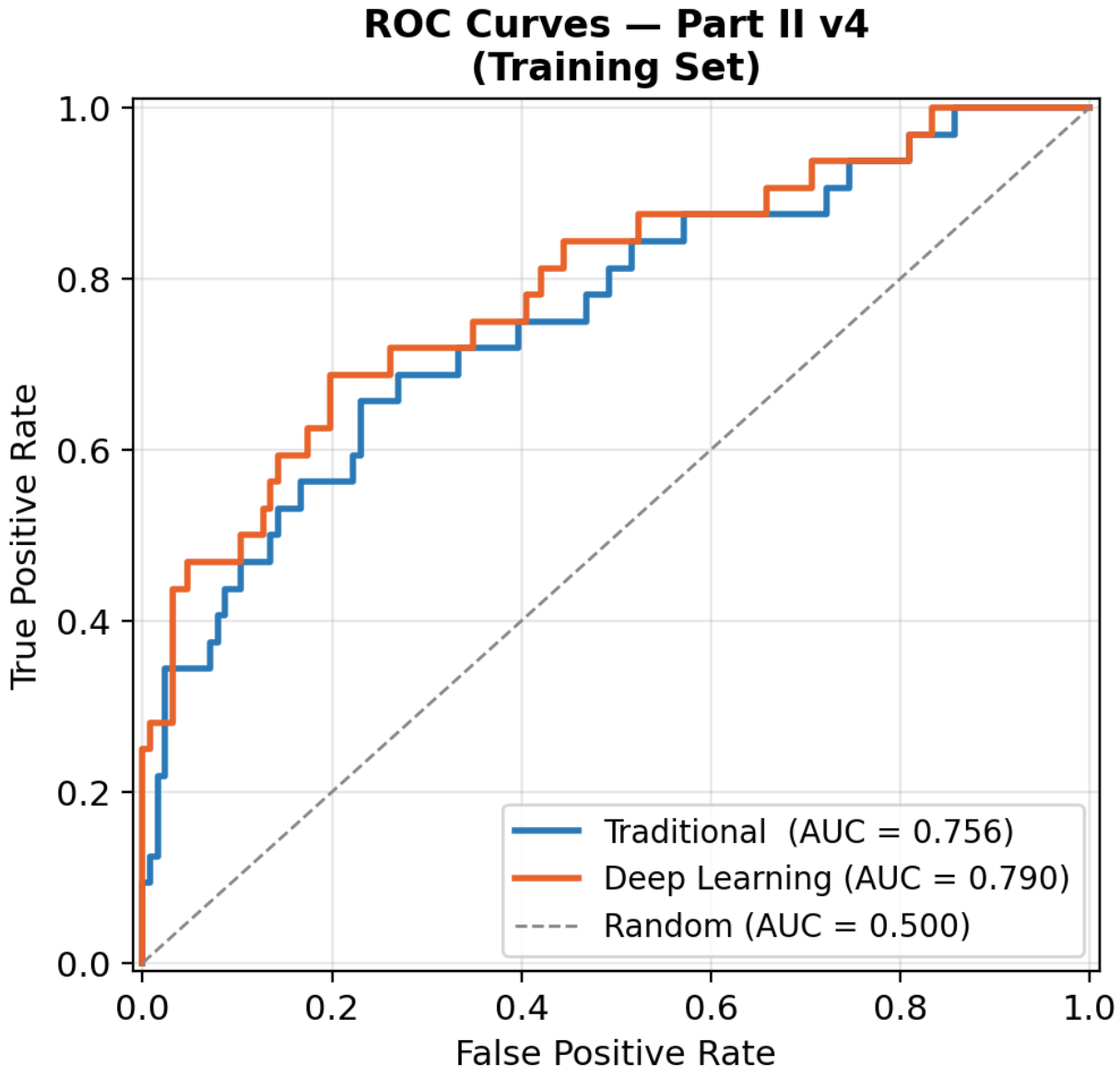

Figure 11: Comparison of ROC curves for both Part II models